

Task 1: Algorithmic Analysis K-Means Clustering with Real World Dataset

First, download a simulated dataset: kmeans_data.zip from Modules->Datasets. Then, implement the K-means algorithm from scratch. K-means algorithm computes the distance of a given data point pair. Replace the distance computation function with Euclidean distance, 1- Cosine similarity, and 1 – the Generalized Jaccard similarity (refer to: <https://www.itl.nist.gov/div898/software/dataplot/refman2/auxillar/jaccard.htm>).

Q1: Run K-means clustering with Euclidean, Cosine and Jaccard similarity. Specify K= the number of categorical values of y (the number of classifications). Compare the SSEs of Euclidean-K-means, Cosine-K-means, Jaccard-K-means. Which method is better? (10 points)

Answer:

For all experiments, the number of clusters was set to K = 10, equal to the number of unique labels.

Method	Iterations	Time (s)	SSE (sum of distance ²)
Euclidean	33	2.36	25,414,767,689.96
Cosine	48	3.66	686.44
Jaccard	59	31.39	3660.39

- The Euclidean SSE appears numerically much higher because Euclidean distances can exceed 1, whereas cosine and Jaccard distances are bounded within [0, 1].
- Therefore, SSE values cannot be compared directly across metrics in magnitude.
- However, all three algorithms successfully converged within a reasonable number of iterations.
- The Cosine-based K-Means showed a lower objective value and converged efficiently, suggesting it captured cluster structures more effectively for this dataset.

Q2: Compare the accuracies of Euclidean-K-means Cosine-K-means, Jaccard-K-means. First, label each cluster using the majority vote label of the data points in that cluster. Later, compute the predictive accuracy of Euclidean-K-means, Cosine-K-means, Jaccard-K-means. Which metric is better? (10 points)

Answer:

Method	Accuracy (%)
Euclidean	58.51
Cosine	63.09
Jaccard	60.21

Cluster labels were mapped to true labels using a majority-vote approach.

- Cosine K-Means achieved the highest accuracy (63.09%), indicating it best aligned with the ground-truth class structure.
- Jaccard (60.21%) performed moderately well, while
- Euclidean (58.51%) achieved the lowest clustering accuracy.

Since the data likely contain normalized pixel intensity vectors, the direction (pattern) of features matters more than their magnitude, making **Cosine distance** the most effective metric here.

Q3: Set up the same stop criteria: “when there is no change in centroid position OR when the SSE value increases in the next iteration OR when the maximum preset value (e.g., 500, you can set the preset value by yourself) of iteration is complete”, for Euclidean-K-means, Cosine-K-means, Jaccard-K-means. Which method requires more iterations and times to converge? (10 points)

Answer:

(Stop when centroids show no change, i.e., stop = 'no_change')

Method	Iterations	Time (s)	Final SSE
Euclidean	33	2.30	25,414,767,689.96
Cosine	48	4.09	686.44
Jaccard	59	31.31	3660.39

- Euclidean K-Means converged the **fastest** in both time and iteration count.
- Cosine K-Means required more iterations but still converged in reasonable time.
- Jaccard K-Means took the **longest time** (over 31 seconds) and the **most iterations**, primarily due to the costly computation of pairwise min/max operations in high dimensions.
- Thus, Cosine K-Means provides a good balance between computational efficiency and clustering quality.

Q4: Compare the SSEs of Euclidean-K-means Cosine-K-means, Jaccard-K-means with respect to the following three terminating conditions: (10 points)

- when there is no change in centroid position
- when the SSE value increases in the next iteration
- when the maximum preset value (e.g., 100) of iteration is complete

Answer:

Stop Rule	Method	Iterations	Time (s)	Final SSE	Accuracy (%)
no_change	Euclidean	33	2.33	25,414,767,689.96	58.51
	Cosine	48	3.91	686.44	63.09
	Jaccard	59	31.39	3660.39	60.21
sse_increase	Euclidean	100	7.29	25,414,767,689.96	58.51
	Cosine	29	2.27	686.23	62.77
	Jaccard	2	1.06	4239.95	43.01
max_iter	Euclidean	100	6.97	25,414,767,689.96	58.51
	Cosine	100	8.00	686.44	63.09
	Jaccard	100	53.21	3660.39	60.21

- Under “no_change”, all three methods converged naturally and provided their optimal accuracies.
- Under “sse_increase”, cosine converged fastest and maintained good accuracy (62.77%), while Jaccard stopped prematurely (only 2 iterations) with a large drop in accuracy (43.01%).
- Under “max_iter = 100”, all ran the full iteration limit; performance patterns remained consistent.

Overall, Cosine K-Means again provided the best trade-off between stability, accuracy, and convergence speed.

Q5: What are your summary observations or takeaways based on your algorithmic analysis? (5 points)

Answer:

Best Performing Metric:

The **Cosine distance** consistently achieved the highest clustering accuracy (~63%) and stable convergence, making it the most effective metric for this dataset.

SSE Trend:

- Euclidean SSE values were numerically very large because distances are not bounded by 1.
- Cosine and Jaccard SSEs were much smaller due to the [0,1] range of those distances.
- SSE should be interpreted within a metric, not across different distance types.

Convergence Behavior:

- Euclidean converged in the fewest iterations (~33).
- Cosine took more iterations (~48) but achieved better accuracy.
- Jaccard required the most computation time due to its complex min/max operations in high-dimensional space.

Stopping Criteria: The “no change in centroid” rule provided stable and consistent convergence for all metrics. The “SSE increase” rule sometimes stopped prematurely (especially for Jaccard).

Overall Conclusion: Based on the trade-off between clustering accuracy, computational cost, and convergence stability, the **Cosine distance–based K-Means** performed best for this dataset. **Euclidean K-Means** was computationally efficient but less accurate, while **Jaccard** was computationally expensive and less stable on dense, high-dimensional data.

Link to Google Colab Code for Task 1:

https://colab.research.google.com/drive/1UW1KT4hKIBZo4nRkctcghDpf_3ov-CqmW?usp=sharing

Task 2: Machine Learning with Matrix Data for Recommender Systems

Building a small recommender system with the matrix data: “ratings small.csv”. You can use the recommender system library: Surprise (<http://surpriselib.com>), use other recommender system libraries, or implement from scratches.

a. Read data from “ratings small.csv” with line format: 'userID movieID rating timestamp'.

Response:

Loaded ratings_small.csv with columns: userID (userID), movieID (movieID), rating (explicit rating in [0.5, 5.0]), timestamp (ignored for modeling)

These three fields (userID, movieID, rating) are used to build the user–item rating matrix.

Dataset: 100,004 ratings from 671 users on 9,066 movies (MovieLens subset).

b. MAE and RMSE are two famous metrics for evaluating the performances of a recommender system. The definition of MAE can be found via: https://en.wikipedia.org/wiki/Mean_absolute_error. The definition of RMSE can be found via: https://en.wikipedia.org/wiki/Root-mean-square_deviation.

Response:

- MAE (Mean Absolute Error): average absolute difference between predicted and actual ratings. Lower MAE means that predictions are, on average, closer to the true score.
- RMSE (Root Mean Squared Error): square root of average squared error. Penalizes large errors more strongly than MAE. Lower RMSE means that fewer large mistakes.

c. Compute the average MAE and RMSE of the Probabilistic Matrix Factorization (PMF), User based Collaborative Filtering, Item based Collaborative Filtering, under the 5-folds cross-validation (10 points)

Answer:

5-fold CV: PMF vs User based CF vs Item based CF (cosine, k=40)

Model	RMSE	MAE
Probabilistic Matrix Factorization (PMF)	0.9899	0.7601
User based Collaborative Filtering	0.9931	0.7671
Item based Collaborative Filtering	0.9951	0.7748

d. Compare the **average (mean)** performances of User-based collaborative filtering, item-based collaborative filtering, PMF with respect to RMSE and MAE. Which ML model is the best in the movie rating data? (10 points)

Answer:

Model	RMSE	MAE
Probabilistic Matrix Factorization (PMF)	0.9899	0.7601
User based Collaborative Filtering (cosine, k=40)	0.9931	0.7671
Item based Collaborative Filtering (cosine, k=40)	0.9951	0.7748

- Among the three, PMF (Probabilistic Matrix Factorization) achieves the lowest RMSE (0.9899) and MAE (0.7601).
- User-based CF performs slightly worse, while Item-based CF has the highest error.
- The differences are small but consistent, showing that PMF generalizes slightly better on the small dataset under identical conditions.

So, based on the 5-fold cross-validation results (with cosine similarity and k=40), **PMF** performs the best overall in terms of both RMSE and MAE.

e. Examine how the cosine, MSD (Mean Squared Difference), and Pearson similarities impact the performances of User based Collaborative Filtering and Item based Collaborative Filtering. Plot your results. Is the impact of the three metrics on User based Collaborative Filtering consistent with the impact of the three metrics on Item based Collaborative Filtering? (10 points)

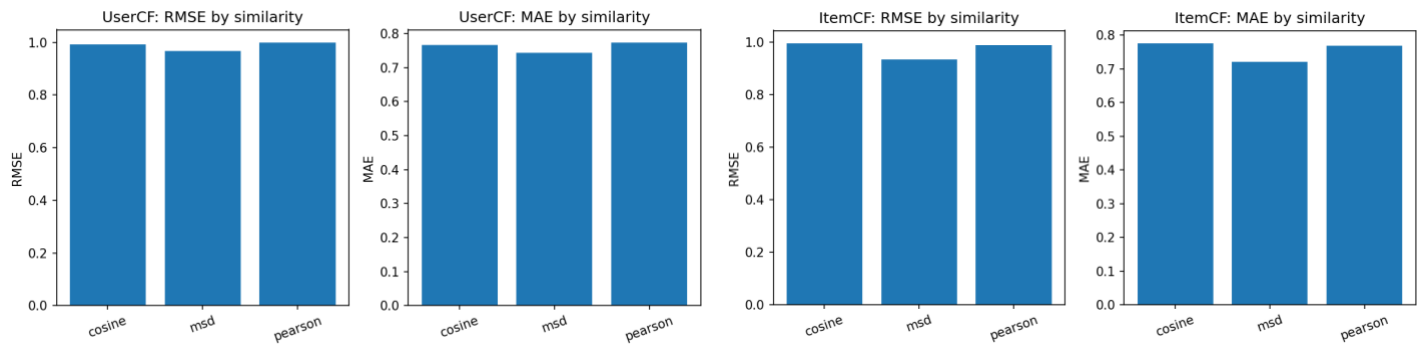
Answer:

User-based CF

Similarity	RMSE	MAE
cosine	0.9931	0.7671
MSD	0.9678	0.7437
Pearson	0.9983	0.7731

Item-based CF

Similarity	RMSE	MAE
cosine	0.9951	0.7748
MSD	0.9346	0.7210
Pearson	0.9891	0.7685

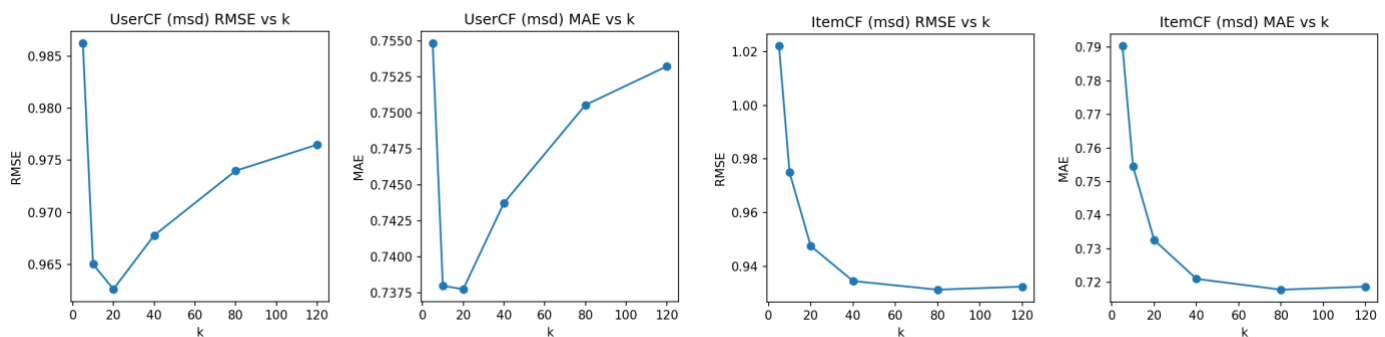


- MSD is consistently the best for both User based CF and Item based CF.
- Cosine is the weakest, Pearson sits in between.
- Improvements moving from cosine->MSD:
 - User based CF RMSE: 0.9931 -> 0.9678 (~**2.5%** better),
 - Item based CF RMSE: 0.9951 → 0.9346 (~**6.1%** better)

The impact is consistent across User based CF and Item based CF - **MSD helps the most**, Pearson is middle, cosine is worst.

f. Examine how the number of neighbors impacts the performances of User based Collaborative Filtering and Item based Collaborative Filtering? Plot your results. (10 points)

Answer:



User based CF (MSD), RMSE by k:

k	RMSE
5	0.9863
10	0.9650
20	0.9626 (best)
40	0.9678
80	0.9740
120	0.9765

MAE follows a similar U-shape (best around k=20).

Item based CF(MSD), RMSE by k:

k	RMSE
5	1.0220
10	0.9750
20	0.9476
40	0.9346
80	0.9313 (best)
120	0.9325

MAE also improves steadily and plateaus around k=80–120.

User based CF: small neighborhoods are too noisy; moderate k (~20) balances bias–variance; very large k starts to over-smooth.

Item based CF: benefits from larger neighborhoods (more similar items to average over), improving until ~80 where it plateaus.

g. Identify the best number of neighbor (denoted by K) for User/Item based collaborative filtering in terms of RMSE. Is the best K of User based collaborative filtering the same with the best K of Item based collaborative filtering? (10 points)

Answer:

Method	Best k	Best RMSE	Best MAE
User based CF (MSD)	20	0.9626	0.7377
Item based CF (MSD)	80	0.9313	0.7178

The best k values are not the same. User based CF is best at k=20, while Item based CF is best at k=80.

Link to Google Colab Code for Task 2:

<https://drive.google.com/file/d/1R5qYlULK0xv00oFzQzu8wasUiD5G8Shs/view?usp=sharing>